Taller de Proyecto II

Informe Final

J5 Brazo Robótico Seguro

Grupo de Desarrollo

- Forden Jones, Ian - 02543/3
- Lugarzo, Matias - 02248/9
- Cuello, Marina Giselle - 02842/2

1. Introducción **3**
2. Objetivo **3**
2.1 Objetivos principales 3
2.2 Requisitos no funcionales 4
3. Esquema gráfico del proyecto **4**
3.1 Distribución general 4
3.2 Conexión de servomotores 5
3.3 Potenciómetro de control 5
3.4 Pulsadores de selección 6
3.5 Fotografías del sistema físico 6
4. Modelo de Anticolisiones **7**
4.1 Modelo geométrico basado en esferas 7
4.2 La matriz de colisiones 8
4.3 Transformaciones y cinemática directa 9
4.4 Cálculo de Rotaciones y Transformaciones en el Modelo Cinemático 10
4.4.1 Construcción de la matriz acumulada 10
4.4.2 Transformación de las esferas locales a coordenadas globales 10
4.5 Detección de colisiones 11
4.6 Integración con el control del brazo 11
5. Interfaz Web **11**
6. Cumplimiento de funcionalidades y requerimientos **13**
6.1 Funcionalidades completamente implementadas 13
6.2 Funcionalidades no implementadas 13
7. Código Fuente y Organización del Proyecto **14**
7.1 algebra_lineal.c 14
7.2 numpy.c 15
7.3 configuracion.c 16
7.4 cinematica.c 18
7.5 main.c 21
8. Replicación e instalación del proyecto **23**
9. Documentación **24**
9.1 Video demostrativo del funcionamiento 24
9.2 Bitácora de desarrollo 24
9.3 Repositorio del código fuente 24

1. Introducción

El presente proyecto se centra en el montaje y uso de un brazo robótico centrado en el desarrollo de su movilidad priorizando la preservación y seguridad de sus componentes.

En experiencias previas de construcción de brazos robóticos, realizadas en el marco de esta asignatura, se identificaron problemas asociados al consumo eléctrico y a las colisiones entre servomotores. Estas dificultades atravesadas evidenciaron la necesidad de contar con mecanismos de control que permitan preservar el material de trabajo.

Actualmente existen librerías que incluyen algoritmos capaces de prever colisiones, su nivel de complejidad y alcance supera los objetivos de esta propuesta. Por lo tanto, se propone implementar un mecanismo de seguridad integrado en el microcontrolador, de modo que sea este el encargado de anticipar y evitar los inconvenientes mencionados mediante cálculos matemáticos simplificados.

De este modo, además del montaje físico del brazo robótico y de la implementación de distintas funcionalidades, la problemática principal que aborda el proyecto consiste en garantizar la seguridad del equipamiento de manera autónoma.

2. Objetivo

2.1 Objetivos principales

- Lograr el movimiento del brazo cambiando el ángulo de los servos mediante el uso de un potenciómetro.
- Incorporar pulsadores que permitan seleccionar qué servomotor se debe accionar.
- Implementar cinemática directa para obtener la pose del efector final a partir de los ángulos de las articulaciones.
- Desarrollar un sistema de detección de colisiones basado en un modelo geométrico simplificado del brazo.
- Proteger los componentes del sistema mediante dicho mecanismo de detección, deteniendo el movimiento ante cualquier riesgo que pudiera dañar los elementos.
- Diseñar una interfaz web que se comunique con el brazo, registre las trayectorias y almacene el estado actual del mismo.
- Incluir en la interfaz web un modo de visualización del brazo robótico mediante un modelo bidimensional o tridimensional.
- Permitir la reproducción de movimientos previamente registrados a través de la interfaz web.

2.2 Requisitos no funcionales

- El sistema debe operar dentro de límites seguros de tensión y corriente para evitar sobrecalentamiento o daños a los servomotores.
- La interfaz web debe ser intuitiva, con botones y controles accesibles sin necesidad de conocimientos técnicos avanzados.
- El sistema debe ser accesible desde navegadores estándar (Chrome, Firefox, Edge) sin necesidad de plugins adicionales.
- El código debe estar documentado y estructurado en módulos para facilitar su actualización y depuración.

3. Esquema gráfico del proyecto

A continuación, en la figura 1, se visualiza el esquema detallado del proyecto con todas las partes físicas involucradas, incluyendo el tipo de conexión que habrá entre ellas.

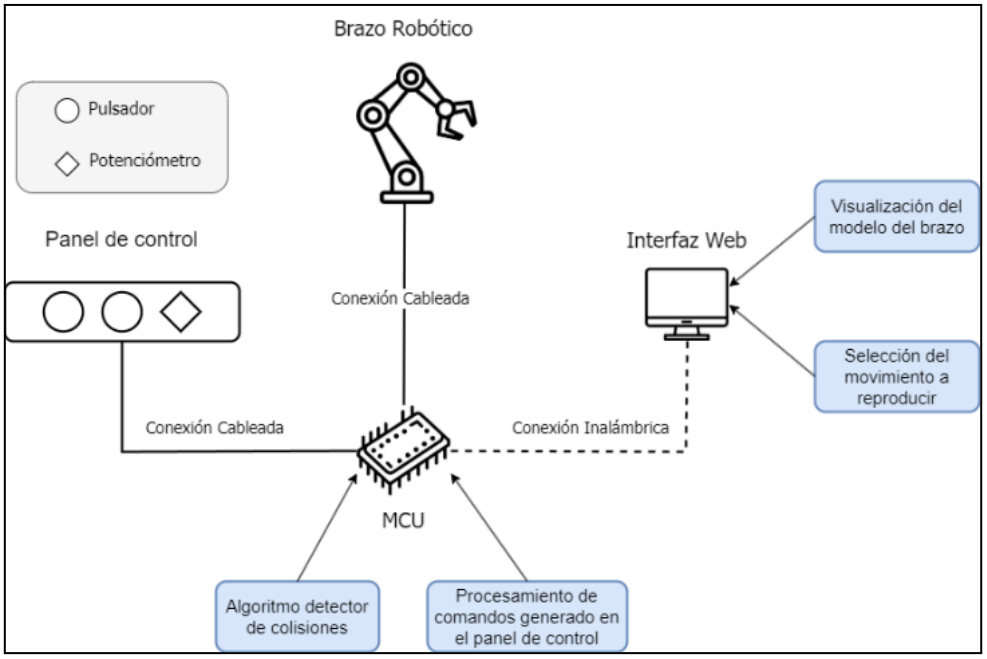

Figura 1. Esquema gráfico del proyecto con las partes que lo componen y sus respectivas funciones principales.

3.1 Distribución general

La alimentación del sistema se realiza mediante una fuente externa de 5 V destinada exclusivamente a los servomotores SG90. Esta separación permite aislar el circuito de potencia del circuito de control, evitando que las variaciones de corriente producidas por los servos introduzcan ruido en las lecturas analógicas del potenciómetro o en el funcionamiento general del microcontrolador.

El ESP32 se alimenta a través del puerto USB conectado a la computadora y utiliza únicamente sus salidas de 3.3 V para alimentar el potenciómetro y los pulsadores de

selección. De este modo, el circuito de control queda completamente independiente de la fuente externa utilizada por los servos.

En este diseño no se comparte la referencia de tierra entre la fuente de 5 V y el ESP32. Las señales PWM que controlan a los servos mantienen su referencia respecto al GND de la fuente externa, mientras que el potenciómetro y los pulsadores operan sobre el GND propio del ESP32. Esta separación fue adoptada con el objetivo de reducir al máximo el ruido eléctrico observado durante las primeras pruebas del sistema.

Para organizar y distribuir las conexiones de forma prolija y segura, se utilizó una protoboard. En ella se separan claramente:

- La línea de 5V y GND correspondientes a los servos (provenientes de la fuente externa).
- La línea de 3.3 V y GND del ESP32 (para potenciómetro y pulsadores).
- Las señales PWM que van del ESP32 hacia cada servomotor.

Esta disposición permitió mejorar la estabilidad del sistema y reducir interferencias durante el funcionamiento.

3.2 Conexión de servomotores

El brazo robótico utiliza cinco servos SG90, cada uno alimentado a 5 V con su respectivo GND común.

Las señales de control PWM se envían desde los siguientes pines del ESP32:

SERVO	GPIO	Pin Físico (ESP32)	Descripción
1	18	D18	Base
2	19	D19	Hombro
3	21	D21	Codo
4	22	D22	Muñeca
5	23	D23	Pinza

Estas conexiones permiten generar señales PWM independientes para cada articulación, garantizando un movimiento fluido y coordinado.

3.3 Potenciómetro de control

El potenciómetro se utiliza para ajustar manualmente el ángulo del servo seleccionado. Se conecta de la siguiente manera:

- Terminal superior → 3.3 V

- Terminal inferior → GND
- Terminal central → GPIO 34 (D34) del ESP32

El microcontrolador interpreta el valor analógico del potenciómetro (0 – 4095) mediante su convertidor ADC interno, transformándolo en un valor angular (0°– 180°) para el servo activo.

3.4 Pulsadores de selección

Se incluyen dos pulsadores externos que permiten desplazarse entre los distintos servomotores a controlar. Ambos se conectan a GND y a los siguientes pines configurados:

Pulsador	GPIO	Pin Físico (ESP32)	Función
Rojo	13	D13	Avanza
Negro	14	D14	Retrocede

3.5 Fotografías del sistema físico

A continuación se incluyen imágenes del brazo robótico, los servomotores, la protoboard utilizada para el cableado y el módulo ESP32:

Figura 2. Estructura completa del brazo robótico con los cinco servomotores SG90 montados.

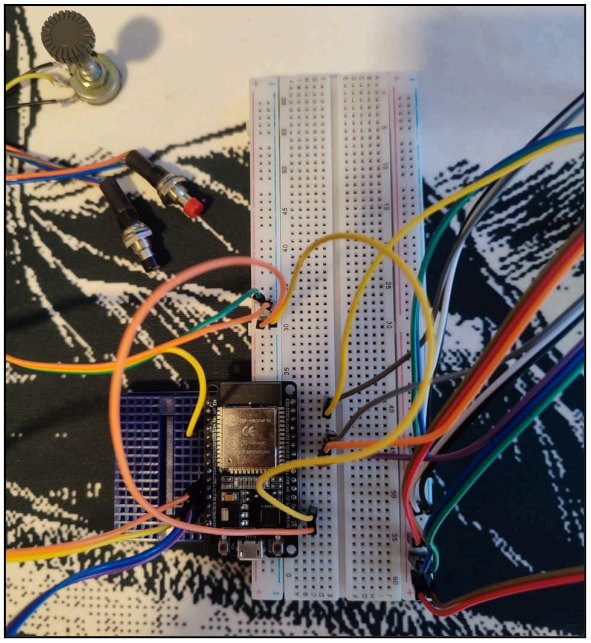

Figura 3. Vista de la protoboard con la distribución de señales

4. Modelo de Anticolisiones

El sistema de anticolisiones del proyecto fue diseñado para prevenir configuraciones peligrosas antes de ejecutar cualquier movimiento en el brazo robótico. Para ello se utiliza un modelo geométrico simplificado basado en esferas, complementado con una matriz de colisiones que indica qué pares de eslabones deben ser evaluados. El cálculo de colisiones se realiza a partir de la cinemática directa, con un modelo matemático optimizado para la ejecución en el ESP32.

4.1 Modelo geométrico basado en esferas

Cada eslabón se aproxima mediante una serie de esferas distribuidas a lo largo de su estructura. Este enfoque reduce el problema de colisiones a comparaciones de distancia entre centros, evitando cálculos geométricos más costosos.

Razones para utilizar este modelo:

- Alta eficiencia computacional.
- Fácil implementación en C.
- Adecuado para microcontroladores con recursos limitados.
- Representación clara en la interfaz web.

La distribución de esferas por eslabón varía según su geometría, permitiendo balancear precisión y velocidad.

4.2 La matriz de colisiones

Para optimizar los cálculos, se definió una matriz de 5×5 que determina qué eslabones pueden colisionar entre sí.

Esta matriz evita evaluar pares de eslabones que nunca interactúan físicamente debido a su posición en la cadena cinemática.

E1	E2	E3	E4	E5
0	0	0	0	1
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
1	1	0	0	0

Su significado es:

- 0: Estos dos eslabones no pueden colisionar, por razones geométricas.
- 1: Estos eslabones sí pueden colisionar, por lo que deben evaluarse todas sus esferas.

¿Por qué se utiliza esta matriz?

- Eficiencia: Sin la matriz, la detección debería evaluar todas las combinaciones posibles (5×5), lo cual implicaría comparar 1–3 esferas por eslabón y multiplicar esa operación muchas veces por segundo.

  La matriz reduce el número de comparaciones necesarias al minimizar pares imposibles.
- Evita falsos positivos:

  Es común que dos eslabones consecutivos compartan un punto en la junta. Evaluarlos podría generar “colisiones” que en realidad son parte normal de la estructura.
- Mejora la claridad del algoritmo: Permite expresar de forma explícita la geometría del brazo y sus relaciones.

La matriz no forma parte del algoritmo matemático, sino que funciona como una regla de ingeniería basada en el diseño físico del brazo.

4.3 Transformaciones y cinemática directa

Para determinar la posición global de cada esfera es necesario aplicar las transformaciones correspondientes a la cinemática directa del brazo.

Cada servo del brazo posee un eje rotacional en Z. Sin embargo, en la geometría original del brazo los ejes no siempre estaban alineados naturalmente al eje Z global. Para corregir esto, se implementó un procedimiento consistente para cada articulación:

1. Aplicar una rotación fija (offset) que reoriente el sistema local del servo de modo que su eje de giro coincida con el eje Z.
2. Aplicar una traslación correspondiente a la distancia física entre la articulación y el inicio del eslabón.
3. Aplicar la rotación dinámica del servo (la que depende del ángulo recibido por el potenciómetro o por la interfaz web).

A continuación se puede observar una imagen de como quedaría el brazo robótico con todos los eslabones con su rotación en Z

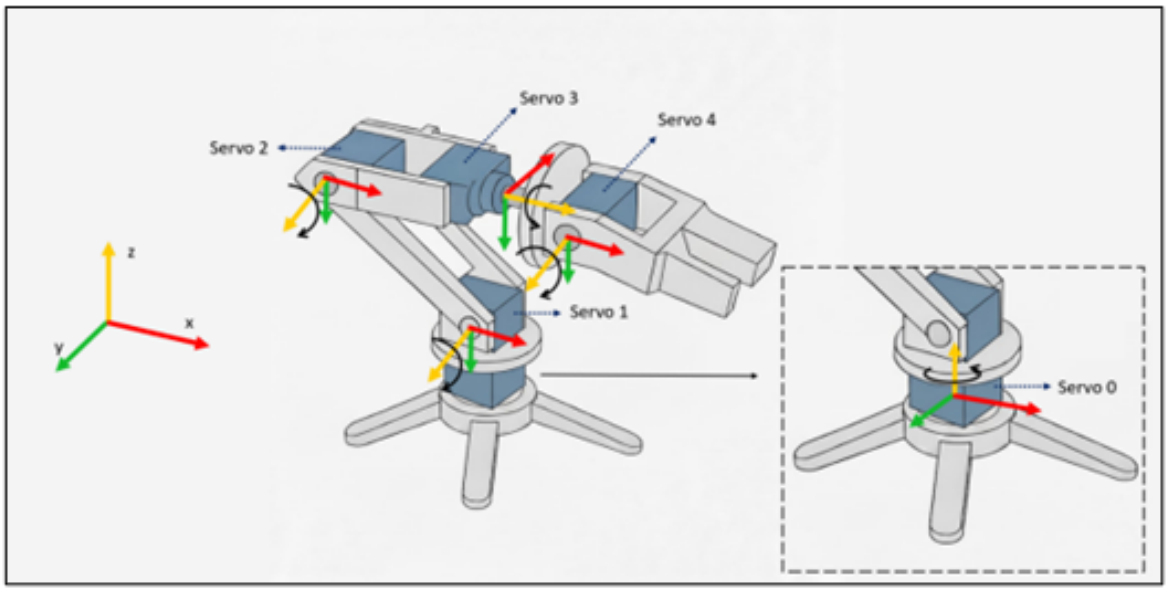

Figura 4

Composición final de la transformación de cada eslabón

Para cada articulación, la matriz acumulada se construye como:

1. Offset de orientación → corrige la dirección del eje de giro.
2. Rotación en Z → ángulo actual del servo.
3. Traslación estructural → largo físico del eslabón.
4. Offset de eslabón → posición local de las esferas del eslabón.

Así, cada esfera local se transforma correctamente al espacio global.

4.4 Cálculo de Rotaciones y Transformaciones en el Modelo Cinemático

El movimiento del brazo robótico se fundamenta en la aplicación de rotaciones y traslaciones en el espacio tridimensional, las cuales permiten determinar la posición y orientación de cada eslabón en una configuración arbitraria. Estas transformaciones son esenciales tanto para la visualización del brazo en la interfaz web como para la evaluación del sistema de anticolisiones, ya que las posiciones globales de las esferas dependen completamente de ellas.

4.4.1 Construcción de la matriz acumulada

La pose global del eslabón i se obtiene multiplicando secuencialmente:

1. Offsets fijos del servo
2. Rotación actual del servo
3. Traslación del eslabón
4. Offset del eslabón (si corresponde)

Es decir:

$$M_i = M_{i-1} * Offset(i) * R(\theta_i) * Traslacion(i)$$

La matriz acumulada comienza como una identidad:

$$M_i = I$$

y se actualiza en cada articulación.

Este procedimiento propaga correctamente la orientación y posición desde la base del brazo hacia el efector final.

4.4.2 Transformación de las esferas locales a coordenadas globales

Cada esfera del eslabón i posee una posición local:

$$P_{local} = (x, y, z)$$

La posición global se obtiene aplicando la matriz acumulada:

$$P_{global} = M_i * P_{local}$$

El resultado son las coordenadas globales reales que se utilizan para:

- visualizar el brazo,
- calcular distancias entre esferas,
- verificar colisiones internas,
- detectar colisión con el plano base.

4.5 Detección de colisiones

Después de transformar todas las esferas, se evalúan:

A) Colisiones internas

Usando únicamente los pares habilitados por la MATRIZ_COLISIONES.

Condición de colisión:

$$dist(A,B) \leq 2 \cdot RADIO_ESFERA$$

B) Colisión contra el plano base

$$z_esfera \leq RADIO_ESFERA$$

Si ocurre cualquiera de estos casos, el movimiento se bloquea antes de ejecutarse.

4.6 Integración con el control del brazo

Antes de mover un servo:

1. Se calcula el ángulo propuesto.
2. Se aplica la cinemática directa completa (incluyendo offsets y reorientaciones en Z).
3. Se evalúa la matriz de colisiones y colisión contra el plano.
4. Si alguna colisión ocurre → se cancela el movimiento.
5. Si la pose es segura → el servo se mueve al nuevo ángulo.
6. Este proceso permite prevenir daños al mecanismo y garantizar un funcionamiento seguro.

5. Interfaz Web

La interfaz web constituye una de las herramientas centrales para la visualización y el monitoreo del estado del brazo robótico. Su objetivo principal es permitir al usuario observar, en tiempo real, la pose del robot, las posiciones globales de las esferas utilizadas en el modelo geométrico y el estado del sistema de anticolisiones, sin necesidad de interactuar directamente con el hardware.

A continuación, se muestra en la Figura 4 la interfaz desarrollada, la cual se comunica directamente con el ESP32 mediante un servidor HTTP interno. Este servidor expone un endpoint que devuelve un objeto JSON con información detallada del estado actual del brazo, incluyendo:

- Los ángulos de cada servomotor.
- La posición global de las esferas que representan cada eslabón.
- El índice del servo activo.
- La indicación de colisión o configuración segura.

La interfaz procesa estos datos y los representa gráficamente mediante un modelo 3D/2D, permitiendo visualizar de forma clara la estructura del brazo y su evolución ante cambios en los ángulos de las articulaciones.

Además de la representación visual, la interfaz cumple funciones complementarias:

- Supervisar el comportamiento del sistema durante las pruebas.
- Detectar rápidamente configuraciones inseguras.
- Verificar que la implementación en el firmware coincida con los resultados del modelo matemático.
- Servir como puente entre el simulador y el brazo físico, permitiendo depurar errores sin necesidad de accionar los servos reales.

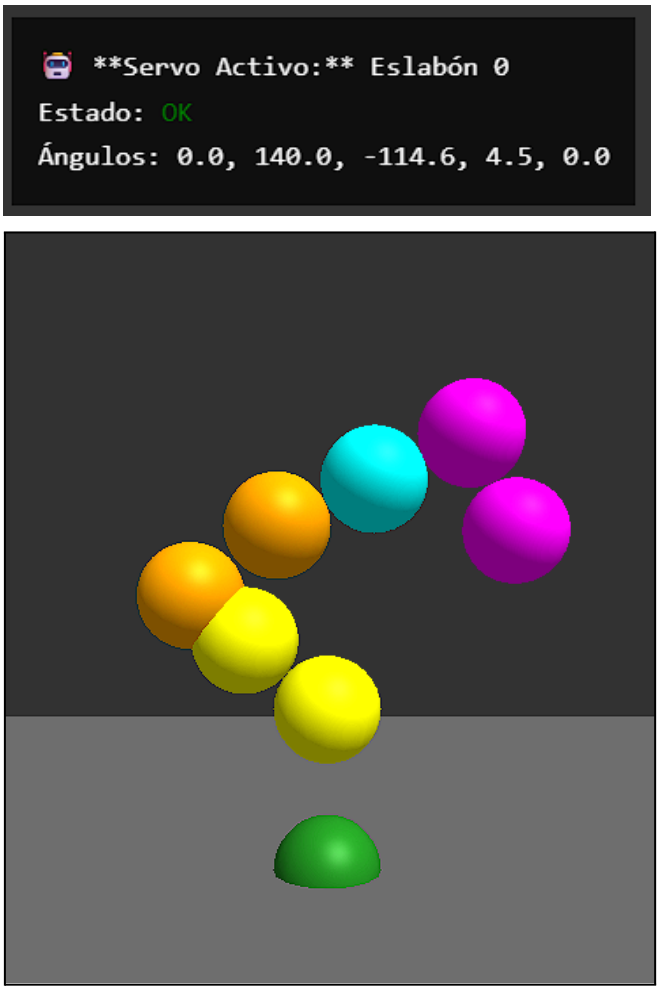

Figura 5

En conjunto, esta interfaz permite validar la cinemática y el sistema de anticolisiones antes de proceder a las pruebas con el hardware real, reduciendo riesgos y acelerando el proceso de desarrollo.

6. Cumplimiento de funcionalidades y requerimientos

6.1 Funcionalidades completamente implementadas

✔ Movimiento del brazo mediante potenciómetro

Se implementó el sistema de lectura analógica con filtrado de señal, mapeo del valor y control progresivo del servo activo. El movimiento resulta estable y reproducible.

✔ Selección de servomotor mediante pulsadores

Los botones NEXT y PREV permiten alternar entre los cinco servos. La selección es confiable y forma parte del flujo de operación principal.

✔ Cinemática directa

Se desarrolló la cinemática mediante matrices de transformación homogéneas, utilizando las rotaciones fijas y longitudes de eslabones definidas en el informe previo .

Permite calcular la pose aproximada del efector final a partir de los ángulos de las articulaciones.

✔ Sistema de anticolisiones

Se implementó el modelo de esferas, el cálculo de distancias entre ellas y la lógica para detener el movimiento ante riesgo de colisión. El chequeo se integra en cada paso del movimiento progresivo.

✔ Home

El brazo inicializa y puede volver a una posición de referencia segura.

✔ Interfaz web funcional + servidor HTTP

Se desarrolló un servidor web en el ESP32 con endpoints JSON que permiten visualizar el estado del brazo.

6.2 Funcionalidades no implementadas

✘ Grabación y reproducción de movimientos

Si bien se implementó la lectura del estado actual del brazo y la comunicación con la interfaz web, no se logró desarrollar el módulo de grabación de trayectorias ni su posterior reproducción, funcionalidades propuestas inicialmente.

7. Código Fuente y Organización del Proyecto

Con el objetivo de garantizar la correcta replicación, continuidad y mantenimiento del proyecto, a continuación se presenta una descripción detallada de la estructura del código fuente desarrollado. Esta sección enumera cada uno de los archivos y módulos que componen el sistema, indicando su función dentro del proyecto y el propósito de las rutinas implementadas en ellos.

El proyecto fue desarrollado siguiendo un enfoque modular, separando las responsabilidades en distintos archivos para mejorar la claridad, el orden y la escalabilidad del código. Cada módulo aborda un aspecto específico del funcionamiento del brazo robótico, ya sea procesamiento matemático, control de servomotores, comunicación con la interfaz web, detección de colisiones o gestión de lectura de sensores.

Siguiendo lo indicado por la cátedra, en esta sección no se incluye código fuente completo, sino una descripción conceptual y funcional de cada archivo. En el caso de archivos de código, se detalla también el objetivo de cada función, permitiendo comprender su rol sin necesidad de inspeccionar el código directamente.

7.1 algebra_lineal.c

Descripción de funciones:

- ***void matriz_rotacion_x(...)***
    - Construye en R una matriz de rotación homogénea 4x4 alrededor del eje X.
    - Convierte el ángulo de grados a radianes y calcula seno y coseno usando las funciones de numpy.h.
    - Deja preparada la matriz para aplicar una rotación pura (sin traslación) sobre cualquier vector 3D en coordenadas homogéneas.
- ***void matriz_rotacion_y(...)***
    - Genera una matriz de rotación 4x4 alrededor del eje Y.
    - Usa el ángulo en grados, lo pasa a radianes y completa los términos de la matriz según la rotación estándar en Y.
    - Se utiliza cuando un eslabón o sistema de referencia del brazo requiere rotar en torno al eje Y.
- ***void matriz_rotacion_z(...)***
    - Crea una matriz de rotación 4x4 alrededor del eje Z.
    - Los términos de la matriz siguen la convención usual de rotación en Z en sistemas homogéneos.
- ***void matriz_traslacion(...)***

    - Inicializa T como matriz identidad 4x4 y luego inserta un vector de traslación (tx, ty, tz) en la última columna.
    - Representa un desplazamiento puro en el espacio, sin rotación.
    - Se usa para avanzar desde un sistema de coordenadas de un eslabón al siguiente según la longitud del eslabón.
- ***void aplicar_transformada(...)***
    - Toma una coordenada 3D local y la convierte a homogénea [x, y, z, 1].
    - Multiplica la matriz de transformación 4x4 por ese vector homogéneo utilizando las funciones de numpy.h.
    - Devuelve en resultado la coordenada 3D transformada (en el sistema global o en el sistema al que corresponda la matriz).
    - Es la función "puente" que aplica la cadena de rotaciones/traslaciones calculadas sobre puntos concretos del brazo (esferas, extremos de eslabones, etc.).

Este módulo agrupa las operaciones básicas de álgebra lineal necesarias para el proyecto: rotaciones alrededor de los tres ejes cartesianos, traslaciones en 3D y aplicación de transformaciones homogéneas sobre puntos.

Sirve como "capa matemática" reutilizable sobre la que se apoyan la cinemática del brazo y el modelo de anticolisiones, evitando repetir lógica de matrices en otros archivos y dejando el resto del código más legible y modular.

7.2 numpy.c

Este archivo implementa una pequeña "capa de utilidades numéricas" inspirada en funciones típicas de NumPy, pero escrita en C, pensada para trabajar con matrices 4x4 y vectores homogéneos usados en la cinemática del brazo.

Descripción de funciones:

- ***double np_cos(...)***
    - Envuelve la función estándar cos de math.h.
    - Recibe un ángulo en radianes y devuelve su coseno.
- ***double np_sin(...)***
    - Envuelve la función estándar sin de math.h.
    - Recibe un ángulo en radianes y devuelve su seno.
- ***void np_eye(...)***
    - Inicializa la matriz M como matriz identidad de tamaño size x size.
    - Para este proyecto se utiliza principalmente con size = 4.
- ***void np_identity(...)***
    - Función equivalente a np_eye, actúa como alias semántico.

    - Se utiliza cuando en el código se quiere reflejar la idea de “identidad” de forma explícita.
- ***void np_append(...)***
    - Convierte una coordenada 3D en un vector homogéneo 4D.
    - Copia x, y, z desde coordenada_local y coloca value como cuarta componente.
    - Permite preparar puntos para ser multiplicados por matrices 4x4.
- ***void np_matmul(...)***
    - Multiplica una matriz 4x4 por un vector 4D: p_global = M @ p_local.
    - Se usa para transformar un punto desde coordenadas locales a globales mediante una matriz homogénea.
    - Es la base de la operación de “aplicar transformada” sobre los puntos del brazo.
- ***void np_matmul_4x4_4x4(...)***
    - Realiza la multiplicación entre dos matrices 4x4: result = A @ B.
    - Se utiliza para encadenar transformaciones en la cinemática.
    - Permite construir la matriz acumulada de cada eslabón a partir de matrices intermedias.

Conclusión general del archivo numpy.c

Este archivo concentra las operaciones numéricas básicas necesarias para trabajar con transformaciones homogéneas en 3D: identidad, armado de vectores homogéneos y multiplicación matriz–vector y matriz–matriz.

Funciona como una capa de abstracción que imita parcialmente el estilo de NumPy, permitiendo que el resto del código (especialmente algebra_lineal.c y cinematica.c) se exprese de manera más clara y cercana al pseudocódigo matemático, sin tener que reescribir constantemente la lógica de estas operaciones de bajo nivel.

7.3 configuracion.c

Este archivo contiene todas las constantes, tablas y parámetros estructurales que definen la configuración física y geométrica del brazo robótico, así como la matriz de colisiones y la inicialización de offsets necesarios para la cinemática y el modelo de seguridad.

Descripción de funciones y constantes:

- ***Constante RADIO_ESFERA***
    - Define el radio de cada esfera usada en el modelo de colisiones.
    - Ajusta el nivel de sensibilidad con el que se detectan proximidades entre eslabones.
- ***Vector T_BASE_GLOBAL***

    - Punto de referencia inicial del brazo en el sistema global.
    - Se utiliza para ubicar el origen del primer sistema de coordenadas.
- ***Matriz MATRIZ_COLISIONES[5][5]***
    - Matriz booleana que indica qué eslabones deben ser evaluados entre sí en la detección de colisiones.
    - 1 = potencial colisión → se deben comparar esferas.
    - 0 = nunca colisionan → se omite el cálculo, optimizando recursos.
    - Evita procesar combinaciones de eslabones imposibles de chocar físicamente.
- ***Vectores de desplazamiento SALTO_0 … SALTO_4***
    - Representan desplazamientos locales entre eslabones, necesarios para construir las transformaciones de cinemática.
    - Cada vector define cómo se mueve el sistema de coordenadas al pasar de un eslabón al siguiente.
- ***Arreglo SALTO_ESLABONES[]***
    - Tabla que permite acceder a los saltos de manera indexada según el número de eslabón.
    - Facilita iterar dinámicamente en algoritmos de cinemática o colisión.
- ***Estructuras esf_0 … esf_4 (esferas locales***)
    - Cada una contiene las coordenadas locales de las esferas asociadas a un eslabón.
    - Representan puntos clave del eslabón, donde se distribuyen esferas digitales para estimar volúmenes.
    - Pueden ser varios puntos por eslabón dependiendo de su geometría.
- ***Arreglo esferas_local[***]
    - Lista genérica de punteros a las estructuras de esferas.
    - Permite acceder a todas las esferas de un eslabón sin importar su cantidad.
    - La detección de colisiones recorre esta estructura.
- ***Matrices de offset T_E0 … T_E4***
    - Representan transformaciones fijas entre la junta y el eslabón correspondiente.
    - Incluyen traslaciones necesarias para posicionar correctamente cada eslabón respecto de la articulación.
- ***Arreglo OFFSET_ESLABON_EN_JUNTA[]***
    - Permite acceder a cada matriz offset según el número de eslabón.
    - Usado en la construcción de las transformaciones completas del brazo.
- ***Función inicializar_matrices_offset()***

- Inicializa todas las matrices de offset entre articulaciones y eslabones.
- Define qué transformaciones fijas deben aplicarse ANTES de la rotación o traslación dinámica del movimiento.
- Acciones principales:
  - T_E0 = identidad.
  - T_E1 y T_E3 se inicializan con traslaciones específicas que reflejan la geometría real del brazo.
  - T_E2 = identidad.
  - T_E4 se inicializa con una traslación distinta, relativa al extremo final.
  - Esta función debe llamarse antes del cálculo de cinemática directa para asegurar que las transformaciones estén correctamente configuradas.

Conclusión general del archivo configuracion.c

Este archivo funciona como el núcleo de parámetros estructurales del brazo robótico.

Define:

- la matriz de colisiones,
- las posiciones locales de las esferas,
- los offsets geométricos,
- vectores de desplazamiento,
- y valores fijos iniciales.

Junto con la función de inicialización, provee todos los datos necesarios para que los módulos de cinemática y detección de colisiones trabajen de manera correcta y consistente. Es un archivo puramente declarativo: no implementa lógica compleja, sino que centraliza la configuración geométrica del brazo para garantizar modularidad y claridad en el resto del proyecto.

7.4 cinematica.c

Este archivo implementa la cinemática directa del brazo robótico, el cálculo de la posición global de las esferas, y toda la lógica de detección de colisiones, tanto internas (entre eslabones) como externas (contra el plano base). También define funciones auxiliares para trabajar con vectores, magnitudes y conjuntos de puntos en colisión.

Descripción de funciones y lógica implementada

- ***np_linalg_norm(...)***
  - Calcula la distancia euclidiana entre dos puntos 3D.
  - Se utiliza para determinar si dos esferas están demasiado cerca entre sí.
- ***ColisionSet_add(...)***
  - Agrega un nuevo centro de esfera al conjunto de colisiones detectadas.

    - Usa realloc para expandir dinámicamente la memoria según se agregan elementos.
    - Es esencial para registrar qué puntos del robot participaron en una colisión.
- ***ColisionSet_union(...)***
    - Une dos conjuntos de centros en colisión en un único set.
    - Se usa al final del proceso para combinar colisiones entre eslabones con colisiones contra el plano.
    - Requiere que result esté inicializado en NULL.
- ***verificar_distancia(...)***
    - Determina si dos esferas chocan comparando su distancia con la suma de radios.
    - Incluye una tolerancia numérica para evitar errores por redondeo.
    - Devuelve true si hay colisión.
- ***detectar_colision_plano_base(...)***
    - Recorre todas las esferas de cada eslabón y detecta si alguna está por debajo de un nivel de Z dado (el plano base).
    - Si una esfera tiene z <= radio + plano_z, se considera colisión.
    - Registra los centros que colisionan en centros_colisionantes_plano.
    - Esta detección evita que el brazo atraviese la mesa o base física.
- ***detectar_colision_par(...)***
    - Detecta colisiones entre dos eslabones específicos.
    - Compara todas las esferas del eslabón A contra las del B.
    - Permite ignorar la esfera que corresponde a la junta en eslabones contiguos (para evitar falsos positivos).
    - Devuelve un ColisionSet que contiene todos los puntos en colisión entre esos dos eslabones.
    - Es la unidad básica para la detección interna del robot.
- ***detectar_colision_total_modular(...)***
    - Ejecuta la detección de colisiones entre todas las combinaciones de eslabones.
    - Usa la MATRIZ_COLISIONES del archivo de configuración para saber qué pares deben verificarse y cuáles se pueden omitir.
    - Combina las colisiones parciales en un conjunto global.
    - Indica si el robot presenta cualquier tipo de colisión interna.
- ***calcular_configuracion_modular(...)***
    - Es la función central del archivo. Realiza:

        Construcción de las matrices de transformación

a. Aplica rotaciones fijas necesarias para alinear ejes.
b. Aplica la rotación de cada servo según el ángulo recibido.
c. Aplica traslaciones de longitud de eslabón (SALTO_ESLABONES).
d. Mantiene una matriz acumulada M_acum para transportar cada transformación.

Transformación de esferas locales a coordenadas globales

e. Para cada eslabón:
f. Reserva memoria con malloc.
g. Aplica la transformada homogénea a cada esfera local.
h. Guarda las esferas globales en esferas_globales[i].
i. Estas coordenadas globales son las que se usarán para detectar colisiones.

Detección de colisión interna

j. Llama a detectar_colision_total_modular.
k. Recibe un conjunto de centros de esferas donde se detectaron colisiones.

Detección de colisión contra el plano base

l. Llama a detectar_colision_plano_base.
m. Identifica si el brazo se hunde bajo el nivel permitido.

Unión de ambos resultados

n. Combina colisiones internas y contra el plano con ColisionSet_union.

Devuelve:

o. true si existe cualquier tipo de colisión.
p. false si la configuración es válida.

Esta función es el núcleo de seguridad del brazo, porque permite verificar si una nueva posición angular es segura antes de ejecutar un movimiento real.

Conclusión general del archivo cinematica.c

Este módulo implementa toda la cinemática directa del brazo robótico y la totalidad del sistema de detección de colisiones.

Es responsable de:

- Calcular posiciones globales de esferas que modelan el volumen de cada eslabón.
- Determinar si el robot se encuentra en una configuración segura.
- Gestionar la memoria de las estructuras de colisión.
- Definir la función central que evalúa una configuración completa del brazo.

En conjunto, este archivo le da al sistema la capacidad de predecir colisiones antes de mover los servos.

7.5 main.c

Descripción de funciones, bloques y lógica implementada:

- **Declaración de pines, constantes y variables globales**
  - Se establecen los pines GPIO para los 5 servomotores.
  - Se configuran también los pines del potenciómetro (ADC) y de los pulsadores NEXT/PREV.
  - Se definen variables globales como:
    - servo seleccionado actualmente,
    - ángulos actuales de cada servo,
    - límites físicos definidos para cada articulación,
    - estados auxiliares (lectura previa, zona muerta, buffers de filtrado, etc.).
  - Se reserva memoria para estructuras usadas en colisiones.
- **Configuración inicial (setup())**
  - Configura los GPIO como entrada o salida según corresponda.
  - Adjunta cada servo a su pin mediante la librería ESP32Servo.
  - Inicializa matrices y estructuras de colisiones con:
    - inicializar_matrices_offset();
  - Configura el módulo ADC del ESP32 para leer el potenciómetro.
  - Inicializa el WiFi y el servidor HTTP, definiendo endpoints (como /estado, /set, etc.).
  - Lleva al brazo a la posición inicial (home) mediante ángulos seguros.
- **Bucle principal (loop())**

  El corazón del comportamiento del brazo. Dentro del loop se ejecutan:
  - Lectura del potenciómetro:
    - Se toma la lectura ADC.
    - Se aplica filtrado: mediana, suavizado o deadzone
    - Se mapea el valor leído a un ángulo válido entre 0° y 180°.
  - Lectura de pulsadores NEXT / PREV:
    - Cambian el servo seleccionado.
    - Previenen rebotes según implementación (con control de tiempo o cambio de estado).
  - Actualización de ángulo objetivo del servo seleccionado:

        - El valor filtrado del potenciómetro se convierte en el nuevo ángulo deseado.
    - Llamado al sistema de seguridad (cinemática + colisión):
        - Se construye un vector de ángulos con los valores actuales y el propuesto.
        - Se llama a: calcular_configuracion_modular(...)
        - Se determina si ese movimiento provocaría una colisión interna o con el plano base.
    - Si NO hay colisión:
        - Se mueve el servo en pequeños pasos usando writeMicroseconds
        - Se actualiza el ángulo actual almacenado en el vector global.
    - Si HAY colisión:
        - NO se ejecuta el movimiento.
        - La interfaz web devuelve un estado indicando colisión.
    - Servidor Web:
        - Dentro del loop también se atienden las peticiones HTTP entrantes.
        - Los endpoints informan el estado actual del brazo o aceptan comandos externos.
- Funciones auxiliares:
    - ***getAngleForServo(...)***: convierte un pulso en microsegundos al ángulo real del servo usando la tabla de calibración correspondiente.
    - ***fallingEdgeDebounced(...***): detecta pulsaciones válidas en los botones aplicando antirrebote por software.
    - ***obtener_pose_global(...***): llama al módulo de cinemática para obtener la pose actual y las colisiones asociadas.
    - ***get_sphere_row_count(...)***: devuelve cuántas esferas tiene cada eslabón para recorrer correctamente los resultados.
    - ***handleDataRequest()***: arma y devuelve en JSON el estado actual del brazo para el frontend web.
    - ***home()***: lleva todos los servos a una posición inicial segura y sincroniza el estado interno.

7.6 Caso de uso: Movimiento seguro de un eslabón mediante potenciómetro

- El usuario gira el potenciómetro para solicitar un cambio de posición en el eslabón activo.
- En loop(), el ESP32 lee el valor analógico, aplica filtrado y lo mapea a un pulso PWM (desiredPulse).
- Se calcula el ángulo propuesto llamando a getAngleForServo(...).

- Se construye un vector de ángulos con la nueva configuración y se llama a calcular_configuracion_modular(...) para obtener la pose global y verificar colisiones.
- Si no se detecta colisión, loop() actualiza el pulso del servo con writeMicroseconds, almacenando el nuevo estado en angulos_grados.
- Si hay colisión, el movimiento no se realiza y se informa la situación por el puerto serie y a través del JSON devuelto al frontend.

Este caso de uso permite vincular la funcionalidad de control manual mediante potenciómetro con los módulos de cinemática y el sistema de anticolisiones.

8. Replicación e instalación del proyecto

1. Instalación del entorno de desarrollo
    a. Instalar Arduino IDE 2.3.6.
    b. Agregar el soporte para placas ESP32 de Espressif Systems 3.3.1.
2. Librerías utilizadas
    a. ESP32Servo (3.0.9): librería utilizada para el control PWM de los servomotores. Instalada mediante el Gestor de Librerías de Arduino.
    b. ArduinoJson (7.4.2): utilizada para la generación y parseo de mensajes JSON en el servidor HTTP embebido. Instalada mediante el Gestor de Librerías.
    c. WiFi.h y WebServer.h: forman parte del core de ESP32 instalado junto con el gestor de placas.
    d. Otras librerías estándar de Arduino utilizadas fueron incluidas a través del propio entorno.

    Todas las librerías adicionales no estándar se entregan junto con el código fuente o se indica claramente cómo instalarlas desde el Gestor de Librerías del IDE.
3. Construcción y carga del firmware
    a. Abrir el proyecto main.ino en el Arduino IDE.
    b. Seleccionar la placa y el puerto serie correspondientes al ESP32
    c. Ajustar, de ser necesario, las credenciales de red (ssid y password) en la sección de configuración.
    d. Compilar y cargar el firmware en la placa desde el IDE.
4. Puesta en marcha
    a. Conectar la fuente de 5V a los servos y el ESP32 por USB al ordenador.
    b. Verificar el conexionado según el esquema de hardware presentado en el informe.

c. Una vez reiniciado el módulo, el ESP32 se conectará a la red WiFi configurada y levantará el servidor HTTP.
d. Desde un navegador, acceder a la IP indicada por el monitor serie para consultar el endpoint /data y la interfaz web de control. Y abrir el archivo index.html para poder visualizar la simulación

9. Documentación

9.1 Video demostrativo del funcionamiento

Se preparó un video de menos de tres minutos que ilustra las funcionalidades principales del brazo robótico, incluyendo:

- Movimiento controlado mediante potenciómetro.
- Selección de servomotores mediante pulsadores.
- Sistema de detección de colisiones basado en cinemática directa.
- Detección de una posible colisión.
- Comunicación con la interfaz web y monitoreo del estado del brazo.

El video se encuentra disponible en el siguiente enlace:
Enlace al video:
Video de funcionamiento final

9.2 Bitácora de desarrollo

Durante el proyecto se mantuvo una bitácora detallada y cronológica que registra los avances, pruebas, decisiones tomadas y dificultades encontradas. Esta documentación fue clave para organizar el trabajo, mejorar el diseño del brazo y fundamentar las modificaciones realizadas durante el desarrollo.

Enlace a la bitácora completa:
Bitácora

9.3 Repositorio del código fuente

El repositorio de control de versiones contiene:

- El firmware completo (main.ino + módulos en C).
- Los scripts y configuraciones adicionales.

Enlace al repositorio del código fuente:
Repositorio en github